

About LOOM: LOOM is an open-source software suite for the automated generation of geographically correct or schematic transit maps, developed by Hannah Bast, Patrick Brosi, and Sabine Storandt at the University of Freiburg (Chair of Algorithms and Data Structures). Source: github.com/ad-freiburg/loom. Licensed under GPL-3.0. This document describes a Windows port produced by Transport for Cairo. All rights in the original LOOM source code remain with the original authors. The Windows-specific patches produced by Transport for Cairo are contributed back under the same GPL-3.0 licence.

1. Project Summary

Transport for Cairo undertook a port of the LOOM transit map generation suite to Windows (MSYS2/UCRT64), enabling the tool to be used on Windows workstations without Docker or WSL. The port involved 10 rounds of iterative patching across approximately 20 source files, resolving POSIX-to-Windows compatibility issues at the C++ source and CMake build system level.

Platform	Status	Method
Windows 10/11 (x64)	Working	MSYS2 UCRT64 + source patches (this document)
macOS (Intel + Apple Silicon)	Working	Native — Homebrew + C++17 flag only
Linux (Ubuntu 20.04+)	Working	Native — apt packages, as per original Dockerfile

2. Windows Build Instructions

2.1 Prerequisites

Install MSYS2 from msys2.org, then open the **MSYS2 UCRT64** shell and run:

```
pacman -S --needed \  
mingw-w64-ucrt-x86_64-cmake \  
mingw-w64-ucrt-x86_64-gcc \  
mingw-w64-ucrt-x86_64-make \  
mingw-w64-ucrt-x86_64-glpk \  
mingw-w64-ucrt-x86_64-coin-or-cbc \  
mingw-w64-ucrt-x86_64-protobuf \  
mingw-w64-ucrt-x86_64-libzip \  

```

```
git
```

2.2 Clone and Apply Patches

```
git clone --recurse-submodules https://github.com/ad-freiburg/loom.git

cd loom

# Apply each patch round in order (PowerShell, from repo root):

.\loom-patches\apply_patches.ps1 # Round 1

.\loom-patches-r2\apply_patches_r2.ps1 # Round 2

# ... through to round 10 (see patch archive)
```

2.3 Build

```
mkdir build && cd build

cmake .. -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release -DLOOM_USE_GUROBI=OFF

mingw32-make -j$(nproc)
```

2.4 Running (use cmd.exe, NOT PowerShell)

PowerShell does not support the < redirection operator for native executables. Always run LOOM commands from cmd.exe or the MSYS2 shell.

Running LOOM

```
# In cmd.exe:

loom.exe < input.json > loom_out.json

transitmap.exe -l < loom_out.json > map.svg

# In MSYS2 shell (full pipeline):

cat examples/stuttgart.json | ./loom | ./transitmap -l > stuttgart.svg
```

2.5 Distributing Binaries (no MSYS2 required on target)

Creating a portable distribution

```
# From build directory in MSYS2:

mkdir -p loom-windows-dist

cp loom.exe topo.exe octi.exe gtfs2graph.exe transitmap.exe topoeval.exe
loom-windows-dist/

cp -r ../examples loom-windows-dist/
```

```
# Bundle required DLLs automatically:

for exe in loom-windows-dist/*.exe; do

  ldd "$exe" | grep -v system32 | grep -v SYSTEM32 | awk '{print $3}' | while read
  dll; do

    [ -f "$dll" ] && cp "$dll" loom-windows-dist/

  done

done
```

3. Complete Patch Registry

All patches are applied to a clean clone of the upstream LOOM repository. No modifications were made to LOOM's core algorithms or data structures — all changes are Windows compatibility shims only.

Round	File	Change
R1	win_compat.h (new file)	Core POSIX shim: unistd.h, dirent.h, isatty, getpid, ssize_t, binary stdio
R1	All *Main.cpp files	Replace #include <unistd.h> with win_compat.h; add win_set_binary_stdio()
R1	CMakeLists.txt	Add WIN32 block: NOMINMAX, C++17, ws2_32/winmm link, include path
R2	win_compat.h	Add netdb.h stub (winsock2/ws2tcpip), pwd.h stub (getpwuid via USERPROFILE)
R2	src/util/log/Log.h	Undefine ERROR/DEBUG/INFO/WARNING before LogLevel enum (Windows macro clash)
R2	src/util/http/Server.cpp	Replace netdb.h with win_compat.h
R2	src/util/Misc.cpp	Replace pwd.h with win_compat.h
R3	win_compat.h	MSYS2 UCRT64 awareness: include system unistd.h first; add uid_t, getpwuid_r, sysconf
R3	win_compat.h	Add pread/pwrite via Windows OVERLAPPED API
R3	src/util/http/Server.cpp	Comment out netinet/in.h, sys/socket.h, arpa/inet.h (covered by winsock2)
R3	src/util/Misc.cpp	Wrap getHomeDir() with #ifdef _WIN32 using USERPROFILE env var
R4	win_compat.h	Fix usleep: inline function instead of macro (avoids Sleep clash)
R4	src/util/http/Server.cpp	Cast setsockopt optval to (const char*); replace SO_REUSEPORT; guard SIGPIPE; def
R4	src/cppgtfs/.../Agency.h	Rename constructor parameter timezone -> tz_ (MinGW time.h macro clash)
R5	src/util/geo/Geo.h	Add #undef Polygon after #include Geo.hpp (wingdi.h clash)
R5	src/cppgtfs/.../Agency.h	Rename member _timezone -> _tz throughout (time.h #define timezone _timezone)
R6	src/util/Misc.h	Add #define NOGDI before #include <windows.h> (prevents wingdi.h)
R6	src/cppgtfs/ (all files)	Rename stop_timezone -> stop_tz and _timezone -> _tz throughout
R7	CMakeLists.txt	Add NOGDI NOUSER NOSOUND to global add_compile_definitions()
R7	src/cppgtfs/Parser.hpp	Fix stop_timezone -> stop_tz in struct flat::Stop references
R8-10	CMakeLists.txt (repair)	Remove broken generator expression fragments left by R8 automation
R10	src/dot/tests/CMakeLists.txt	Remove -lutil (POSIX-only, not needed); restore ad_cppgtfs as separate token
R10	src/gtfs2graph/tests/CMakeLists.txt	Same as above
R10	src/octi/tests/CMakeLists.txt	Same as above
R10	src/util/protobuf/Protobuf.h	Guard #include <arpa/inet.h> with #ifdef _WIN32 / winsock2.h
R10	src/transitmap/output/MvtRenderer.cpp	Add #include <direct.h>; fix mkdir() call (Windows takes 1 arg, not 2)

4. QGIS Plugin — Project Brief

This section serves as the project brief and context document for the next phase of work: building a cross-platform QGIS plugin that wraps LOOM with a friendly GUI, abstracting away all command-line usage.

4.1 Objective

Build a QGIS plugin (Python) that allows transit planners and GIS analysts to generate schematic and geographically accurate transit maps directly inside QGIS, without any knowledge of the LOOM command-line interface. The plugin should work identically on Windows, macOS, and Linux.

4.2 LOOM Pipeline Overview

The full LOOM pipeline reads/writes GeoJSON via stdin/stdout:

```
gtfs2graph -m tram feed.zip # GTFS -> GeoJSON line graph
| topo # resolve overlapping edges
| loom # optimise line orderings
| octi [-b orthoradial] # schematise (optional)
| transitmap -l # render to SVG
> output.svg
```

4.3 Plugin Architecture

Component	Description
Binary resolver	Detects OS at runtime; points to bundled Windows .exe + DLLs, or system binaries on Linux/Mac
Pipeline runner	Executes LOOM tools as subprocesses using subprocess.run() with stdin/stdout piping; no shell redirection
Input layer handler	Accepts GeoJSON or GTFS input from QGIS layer or file picker; converts to LOOM-compatible format
Parameter UI	Qt dialog with controls for: transport mode, line width, spacing, render style, output format
Output handler	Loads SVG output into QGIS as a annotation layer, or saves to file; optionally loads GeoJSON intermediate
Progress reporting	QgsTask-based async execution with progress bar; non-blocking UI

4.4 OS Detection and Binary Strategy

```
Binary resolver (Python)

import platform, os, sys

def get_loom_binaries():

    system = platform.system() # 'Windows', 'Darwin', 'Linux'

    plugin_dir = os.path.dirname(__file__)
```

```

if system == 'Windows':

    bin_dir = os.path.join(plugin_dir, 'bin', 'windows')

    ext = '.exe'

elif system == 'Darwin':

    bin_dir = os.path.join(plugin_dir, 'bin', 'macos')

    ext = ''

else:

    bin_dir = os.path.join(plugin_dir, 'bin', 'linux')

    ext = ''

return {

    'loom': os.path.join(bin_dir, f'loom{ext}'),

    'topo': os.path.join(bin_dir, f'topo{ext}'),

    'octi': os.path.join(bin_dir, f'octi{ext}'),

    'gtfs2graph': os.path.join(bin_dir, f'gtfs2graph{ext}'),

    'transitmap': os.path.join(bin_dir, f'transitmap{ext}'),

}

```

4.5 Subprocess Pipeline (no shell required)

On Windows, never use `shell=True` or pipe via `cmd.exe`. Use `subprocess.PIPE` to chain processes directly in Python — this avoids all PowerShell/cmd redirection issues and works identically on all platforms.

Pipeline runner (Python)

```

import subprocess

def run_pipeline(input_geojson: str, cfg: dict) -> str:

    bins = get_loom_binaries()

    data = input_geojson.encode('utf-8')

    # Stage 1: loom (line ordering optimisation)

    p1 = subprocess.run([bins['loom']], input=data,

        capture_output=True)

    # Stage 2: octi (optional schematisation)

    if cfg.get('schematic'):

```

```

base = cfg.get('base_graph', 'octilinear')

p2 = subprocess.run([bins['octi'], '-b', base],
input=p1.stdout, capture_output=True)

render_input = p2.stdout

else:

render_input = p1.stdout

# Stage 3: transitmap (SVG rendering)

args = [bins['transitmap']]

if cfg.get('labels'): args.append('-l')

p3 = subprocess.run(args, input=render_input, capture_output=True)

return p3.stdout.decode('utf-8')

```

4.6 Suggested UI Parameters

Parameter	UI Control	LOOM flag / notes
Input file / layer	File picker + layer dropdown	GeoJSON or GTFS zip
Transport mode	Dropdown (tram/bus/rail/all)	-m flag for gtfs2graph
Render style	Radio (geographic / octilinear / orthographic)	transitmap octi -b flag
Show labels	Checkbox	transitmap -l
Line width	Spinner (px)	transitmap --line-width
Line spacing	Spinner (px)	transitmap --line-spacing
Outline width	Spinner (px)	transitmap --outline-width
Output format	Dropdown (SVG / MVT tiles)	transitmap --render-engine
Output path	File save picker	Written after pipeline
ILP solver	Dropdown (auto/glpk/cbc)	loom --ilp-solver

4.7 Suggested Repository Structure

```

qgis-loom-plugin/
├── plugin/
│   ├── __init__.py
│   ├── loom_plugin.py # QGIS plugin entry point
│   └── dialog.py # Qt UI definition

```

- ■■■ runner.py # subprocess pipeline runner
- ■■■ binary_resolver.py # OS detection + binary paths
- ■■■ bin/
- ■ ■■■ windows/ # .exe files + bundled DLLs
- ■ ■■■ macos/ # compiled binaries
- ■ ■■■ linux/ # compiled binaries
- ■■■ resources/
- ■■■ icon.png
- patches/ # All Windows source patches (R1-R10)
- build_scripts/ # Scripts to build binaries per OS
- README.md
- LICENSE # GPL-3.0 (matching upstream LOOM)

5. Licence & Attribution

5.1 Original LOOM Tool

LOOM is copyright © University of Freiburg, Chair of Algorithms and Data Structures. Authors: Hannah Bast, Patrick Brosi, Sabine Storandt. Published under the GNU General Public Licence v3.0 (GPL-3.0).

Source repository: <https://github.com/ad-freiburg/loom>

Key publications:

- Bast, Brosi, Storandt. *Efficient Generation of Geographically Accurate Transit Maps*. SIGSPATIAL 2018.
- Bast, Brosi, Storandt. *Metro Maps on Octilinear Grid Graphs*. EuroVis 2020.
- Bast, Brosi, Storandt. *Metro Maps on Flexible Base Grids*. SSTD 2021.

5.2 Transport for Cairo Contributions

The Windows compatibility patches described in this document were produced by Transport for Cairo (transportforcairo.com) in 2026. These patches are contributed under the same GPL-3.0 licence as the upstream LOOM project.

Transport for Cairo is a Cairo-based urban mobility consultancy specialising in public transport planning, data analysis, and transit technology for cities in the Global South.

5.3 Suggested Attribution Text

This tool uses LOOM (github.com/ad-freiburg/loom), developed by Hannah Bast, Patrick Brosi, and Sabine Storandt at the University of Freiburg, licensed under GPL-3.0. Windows port by Transport for Cairo (transportforcairo.com), 2026.